H04 Database Integration

TypeORM vs Prisma pada Social API (NestJS)

Bara S. Rohmani · NRP 5025241144 · Teknik Informatika ITS

1 Pendahuluan

Tugas ini membangun API sosial mirip Twitter (users, posts, likes) dua kali dengan NestJS dan SQLite: satu dengan TypeORM (`/typeorm/*`, database `ecommerce.sqlite`) dan satu dengan Prisma (`/prisma/*`, database `prisma/dev.db`). Kedua implementasi memakai DTO dan `ValidationPipe` global yang sama, sehingga yang dibandingkan hanya cara mengakses database.

2 Implementasi TypeORM

2.1 Langkah

1. Mengisi entity `User`, `Post`, dan `Like` beserta relasinya.
2. Mengisi DTO create; DTO update tetap `PartialType`.
3. Mengisi service (create, findAll, findOne, update, remove) untuk ketiga resource.
4. Menjalankan test e2e. Test baru dapat dijalankan setelah schema Prisma di-generate (lihat bagian Prisma), karena `afterEach` memanggil `prisma.*` bahkan untuk test TypeORM.

2.2 Keputusan utama

Entity. Satu user memiliki banyak post dan like; satu post memiliki banyak like. Relasi `ManyToOne` memakai `onDelete: 'CASCADE'`, dan like diberi `@Unique(['userId', 'postId'])` agar satu user tidak dapat menyukai post yang sama dua kali. Kolom FK (`authorId`, `userId`, `postId`) ditulis eksplisit dengan `@Column()` di samping relasinya, karena DTO menerima ID numerik dan PATCH like mengubah `userId` secara langsung.

```
@Entity('likes')
@Unique(['userId', 'postId'])
export class Like {
  @PrimaryGeneratedColumn()
  id!: number;

  @Column()
  userId!: number;

  @Column()
  postId!: number;

  @ManyToOne(() => User, (user) => user.likes, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'userId' })
  user!: User;

  @ManyToOne(() => Post, (post) => post.likes, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'postId' })
  post!: Post;
}
```

DTO. DTO create memakai decorator `class-validator` (`@IsString`, `@IsNotEmpty`, `@IsEmail`, `@IsInt`). DTO update cukup `PartialType`, sehingga aturan validasi diwarisi dan semua field

menjadi opsional. Karena `forbidNonWhitelisted: true`, field tanpa decorator ditolak dengan 400, jadi setiap field DTO wajib didekorasi.

Service. `findOne` melempar `NotFoundException` bila record tidak ada; `update` dan `remove` mencari record lebih dulu agar respons 404 benar. `update` sengaja tidak memuat relasi, agar objek relasi yang termuat tidak bertabrakan dengan perubahan FK pada PATCH like. `LikesService.create` memeriksa duplikat `(userId, postId)` dengan satu query sebelum `save` dan melempar `ConflictException`; `@Unique` pada entity tetap menjadi pengaman terakhir.

2.3 Kendala yang dialami

- **Native binding SQLite.** TypeORM melaporkan package SQLite tidak memiliki native binding. `sqlite3` sudah ada di dependency, tetapi pnpm memblokir build script-nya. Setelah build script disetujui dan package dibangun ulang, test e2e dapat berjalan.
- **Error `ts(2564)`.** Properti entity dan DTO ditandai "has no initializer" karena `strictPropertyInitialization` aktif, padahal nilainya diisi framework saat runtime. Solusinya definite-assignment assertion (`!`) pada setiap properti; ini juga dipakai pada DTO Prisma.

3 Implementasi Prisma

3.1 Langkah

1. Mengisi model `User`, `Post`, dan `Like` di `schema.prisma`.
2. Menjalankan `pnpm prisma db push` lalu `pnpm prisma generate`.
3. Menyalin DTO dari TypeORM (isinya identik).
4. Mengisi service untuk ketiga resource.
5. Menjalankan seluruh test e2e.

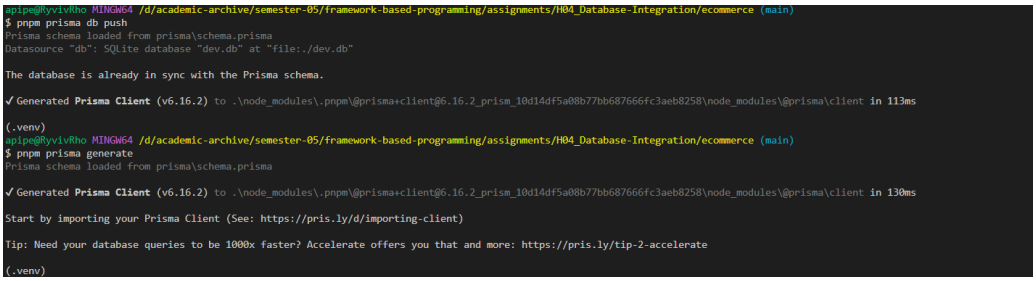
3.2 Keputusan utama

Schema. Model dibuat sejajar dengan entity TypeORM: `email` unik, relasi `onDelete: Cascade`, `@@unique([userId, postId])`, dan `@@map` ke `users`, `posts`, `likes`. `db push` menyelaraskan database dengan schema, sedangkan `generate` membangkitkan client dan tipe TypeScript. Keduanya harus dijalankan sebelum test apa pun, termasuk test TypeORM, karena `afterEach` memanggil `prisma.like.deleteMany()` dan sejenisnya.

```
model Like {
  id     Int @id @default(autoincrement())
  userId Int
  postId Int

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  post Post @relation(fields: [postId], references: [id], onDelete: Cascade)

  @@unique([userId, postId])
  @@map("likes")
}
```



Gambar 1: Output `pnpm prisma db push` dan `pnpm prisma generate`.

Service. Service memakai `create`, `findMany`, `findUnique`, `update`, dan `delete`, dengan `include` untuk relasi (user → posts; post → author dan likes; like → user dan post). `findUnique` yang mengembalikan `null` diubah menjadi `NotFoundException`. `update` dan `delete` Prisma melempar error `P2025` bila record tidak ada, sehingga cukup satu query lalu `P2025` ditangkap menjadi 404. `P2002` (pelanggaran unique) pada user dan like dipetakan menjadi `ConflictException`, dan `PostsService` memetakan `P2003` (FK tidak valid) menjadi 400.

3.3 Kendala yang dialami

Editor menampilkan `Module '"@prisma/client"' has no exported member 'Prisma'` pada import yang dipakai untuk memeriksa `PrismaClientKnownRequestError`. Diagnostik hilang setelah TS Server di-restart (`Ctrl+Shift+P` → `TypeScript: Restart TS Server`). Sebelumnya `prisma generate` sempat gagal dengan `EPERM` saat mengganti file query engine. Penyebab pasti kedua hal ini belum saya ketahui.

4 Pengujian dan Menjalankan Aplikasi

4.1 Menjalankan server

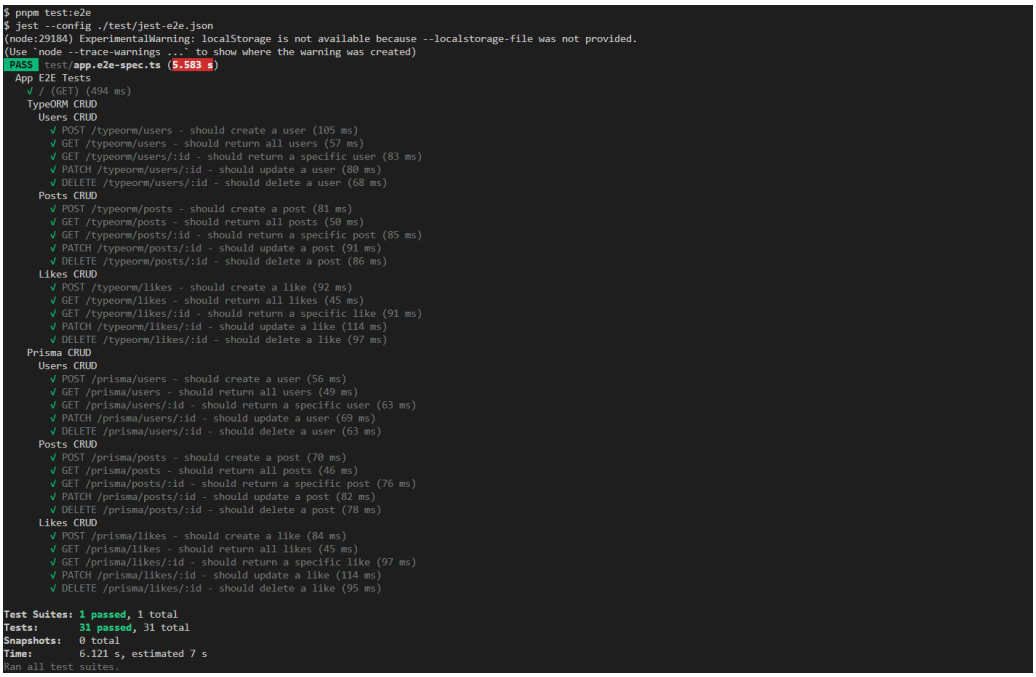
Server dijalankan dengan `pnpm start:dev`. Log menunjukkan route `/typeorm/*` dan `/prisma/*` terpetakan dan aplikasi berhasil start tanpa error.



Gambar 2: Log `pnpm start:dev`: route terpetakan dan aplikasi berjalan.

4.2 Test e2e

Test dijalankan dengan `pnpm test:e2e`. Hasil terakhir: 1 suite dan 31 test lulus (1 root, 15 TypeORM, 15 Prisma). Test mengirim request HTTP lewat Supertest dan membersihkan kedua database pada `afterEach`. `pnpm tsc --noEmit` dan `pnpm run build` juga berhasil.



Gambar 3: Hasil `pnpm test:e2e`: seluruh test TypeORM dan Prisma lulus.

4.3 Uji manual endpoint

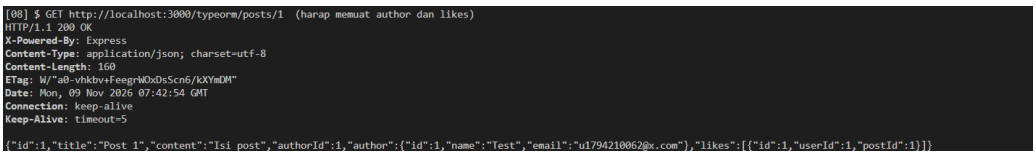
Selain test otomatis, endpoint dicoba manual untuk melihat respons nyata. Urutannya: membuat user, membuat post dengan `authorId` user tersebut, membuat like, lalu membaca post. Pasangan gambar berikut menunjukkan TypeORM (kiri) dan Prisma (kanan) dengan request yang sama.



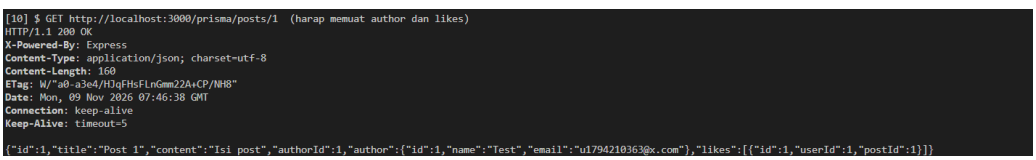
Gambar 4: TypeORM: `POST /typeorm/users` mengembalikan 201.



Gambar 5: Prisma: `POST /prisma/users` mengembalikan 201.



Gambar 6: TypeORM: `GET /typeorm/posts/:id` memuat `author` dan `likes` lewat `relations`.



Gambar 7: Prisma: `GET /prisma/posts/:id` memuat `author` dan `likes` lewat `include`.

Like yang sama dikirim dua kali. Keduanya menolak dengan 409, tetapi lewat mekanisme berbeda: TypeORM memeriksa lebih dulu di service, Prisma menangkap `P2002` dari database. Akses ke ID yang tidak ada mengembalikan 404 pada keduanya.



Gambar 8: TypeORM: like duplikat ditolak 409.



Gambar 9: Prisma: like duplikat ditolak 409.



Gambar 10: TypeORM: `GET /typeorm/users/9999` mengembalikan 404.



Gambar 11: Prisma: `GET /prisma/users/9999` mengembalikan 404.

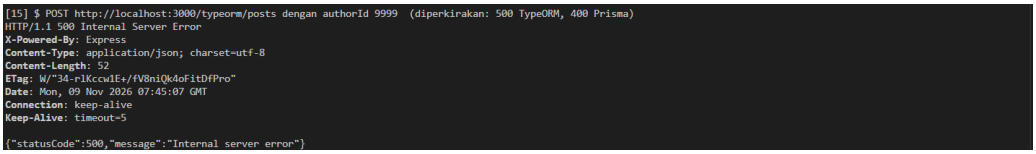
5 Perbandingan TypeORM vs Prisma

Aspek	**TypeORM**	**Prisma**
Definisi model	Class entity dengan decorator di kode aplikasi; tabel dibuat otomatis oleh `synchronize: true`.	File `schema.prisma` terpisah; tabel dibuat lewat `db push`.
Type safety	Tipe berasal dari class entity. Properti perlu `!` (`ts(2564)`) karena diisi ORM saat runtime.	Client bertipe dibangkitkan dari schema. Sebelum `generate`, `Prisma` dan `prisma.like` belum tersedia.
Relasi	Opsi `relations` pada `find`; harus dihindari di `update` agar tidak bentrok dengan FK.	`include` pada query baca; `update` dengan `data: { userId }` langsung.
Data tidak ditemukan	`findOne` dulu, `null` menjadi `NotFoundException` (dua query untuk update/remove).	`update`/`delete` satu query, lalu tangkap `P2025`.
Duplikat like	Cek `findOneBy` sebelum `save` (dua query). Secara teori, dua request bersamaan dapat	Biarkan database menolak, tangkap `P2002` menjadi 409 (satu query, atomik).

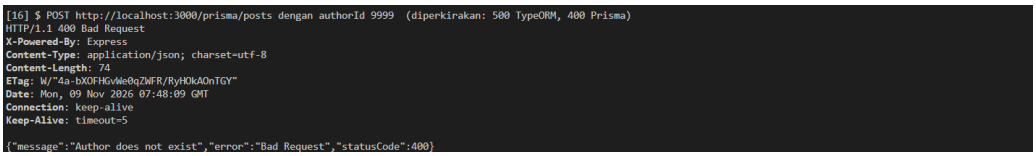
Aspek	TypeORM	Prisma
	sama-sama lolos dan baru ditolak constraint database.	
Setup dan kendala nyata	Tanpa langkah generate. Kendala: native binding `sqlite3` diblokir pnpm.	Wajib `db push` dan `generate`. Kendala: diagnostik editor `Prisma` dan `EPERM`.

5.1 Batasan implementasi saya

- Test e2e hanya menguji jalur sukses CRUD. Payload tidak valid, ID tidak ada, duplikat email atau like, FK tidak valid, dan cascade belum diuji, sehingga keberadaan handler di kode belum membuktikan kasus itu bekerja.
- FK tidak valid pada TypeORM tidak dipetakan menjadi exception HTTP khusus. Pada Prisma, `PostsService` memetakan `P2003` menjadi 400 tetapi `LikesService` belum, sehingga perilakunya tidak konsisten.
- `UsersModule`, `PostsModule`, `LikesModule`, dan `PrismaModule` masing-masing mendaftarkan `PrismaService`, sehingga ada beberapa instance client ke file database yang sama. Cukup untuk SQLite lokal, tetapi bukan desain produksi.



Gambar 12: TypeORM: `POST /typeorm/posts` dengan `authorId` tidak ada menghasilkan 500.



Gambar 13: Prisma: `POST /prisma/posts` dengan `authorId` tidak ada menghasilkan 400.

6 Verdict

Pilihan: Prisma.

Konteks: Proyek tim kecil dengan schema yang sering berubah dan kebutuhan CRUD biasa. Penilaian ini berdasarkan satu tugas kecil (SQLite, CRUD sederhana); query kompleks dan performa belum saya uji.

Alasan:

1. Penanganan error lebih ringkas: `update`/`delete` cukup satu query dengan `P2025`, dan duplikat like diputuskan database secara atomik lewat `P2002`. Di TypeORM, keduanya memerlukan `findOne` atau pemeriksaan manual lebih dulu.
2. Schema menjadi satu sumber definisi dan tipe client dibangkitkan darinya. Di TypeORM, FK harus dideklarasikan dua kali (`@Column` dan relasi) agar PATCH like bekerja.

Kelemahan pilihan saya: Langkah `generate` wajib dan menjadi sumber kendala nyata (diagnostik `Prisma` di editor, `EPERM`), serta kode error Prisma harus dipetakan sendiri ke respons HTTP. TypeORM lebih sesuai jika tim ingin entity berupa class di kode aplikasi dan tidak ingin langkah build tambahan.

7 Penutup

Hal yang paling saya pelajari adalah bagaimana tiap ORM menangani error dan relasi secara berbeda untuk hasil CRUD yang sama. Bagian paling menantang justru tooling (build native `sqlite3`, `generate`), bukan penulisan CRUD-nya. Pada proyek berikutnya saya akan menambah test untuk jalur error (404, 409, FK tidak valid) sejak awal.